CDI College

BDA400 - Data Science Tools and Techniques

Assignment 5

Technical Analysis Using R - Development Phase

Student: Parvana Babali

Assignment Value: 15%

Submission Filename: Babali_Parvana_CA_BDA400_A05.pdf

Repository: https://github.com/ParvanaB/TechnicalAnalysis

Repository Folder: Assignment05

1. Assignment Overview

This development-stage submission implements the nine technical-analysis functions required by the assignment using standard core R operations. Each indicator is stored in its own R script with the exact function name specified by the template. A separate test script demonstrates the functions, and an integration script shows how to apply them to stock-price CSV files produced in the preliminary stage.

2. File Organization and Running Instructions

```
TechnicalAnalysis/
└── Assignment05/
    ├── sma.R
    ├── ema.R
    ├── macd.R
    ├── stdev.R
    ├── linreg.R
    ├── rsi.R
    ├── stoch_rsi.R
    ├── crossover.R
    ├── crossunder.R
    ├── test_indicators.R
    ├── apply_to_stock_data.R
    ├── README.md
    └── Babali_Parvana_CA_BDA400_A05.pdf
```

To run all sample tests in RStudio:

```
setwd("path/to/TechnicalAnalysis/Assignment05")
source("test_indicators.R")
```

To apply the functions to an Assignment 2 stock CSV:

```
source("apply_to_stock_data.R")
results <- analyze_stock_file("AAPL_data.csv")
print(results)
```

3. Simple Moving Average (SMA)

The function validates the numeric vector and period, then slides a fixed-size window across the data. Each output is the sum of the current window divided by the period. This follows the supplied SMA formula and pseudocode.

```
# BDA400 Assignment 5 - Simple Moving Average (SMA)
# Student: Parvana Babali
# Base R only - no indicator libraries are used.

sma <- function(data, period) {
  if (!is.numeric(data)) stop("data must be a numeric vector")
  if (length(period) != 1 || !is.numeric(period) || is.na(period) ||
      period <= 0 || period != as.integer(period)) {
    stop("period must be one positive integer")
  }
  if (anyNA(data)) stop("data must not contain NA values")
  if (length(data) < period) {
    stop("Data length should be greater than or equal to the period")
  }

  sma_values <- numeric(length(data) - period + 1)
```

```
  for (i in seq_len(length(sma_values))) {
    current_window <- data[i:(i + period - 1)]
    sma_values[i] <- sum(current_window) / period
  }

  return(sma_values)
}
```

4. Exponential Moving Average (EMA)

The EMA uses the assignment multiplier 2/(period + 1). The first EMA equals the first data point; every later value gives greater weight to the current observation using the recursive EMA formula.

```
# BDA400 Assignment 5 - Exponential Moving Average (EMA)
# Student: Parvana Babali
# Base R only - no indicator libraries are used.

ema <- function(data, period) {
  if (!is.numeric(data)) stop("data must be a numeric vector")
  if (length(period) != 1 || !is.numeric(period) || is.na(period) ||
      period <= 0 || period != as.integer(period)) {
    stop("period must be one positive integer")
  }
  if (length(data) == 0) stop("data must contain at least one value")
  if (anyNA(data)) stop("data must not contain NA values")

  multiplier <- 2 / (period + 1)
  ema_values <- numeric(length(data))

  ema_values[1] <- data[1]

  if (length(data) > 1) {
    for (i in 2:length(data)) {
      ema_values[i] <- (data[i] - ema_values[i - 1]) * multiplier +
        ema_values[i - 1]
    }
  }

  return(ema_values)
}
```

5. Moving Average Convergence Divergence (MACD)

The function calculates short and long EMAs with the student-written ema() function, subtracts them to obtain the MACD line, computes an EMA signal line, and subtracts signal from MACD to create the histogram.

```
# BDA400 Assignment 5 - Moving Average Convergence Divergence (MACD)
# Student: Parvana Babali
# Depends only on the student-written ema() function in ema.R.

macd <- function(data, short_period, long_period, signal_period) {
  if (!is.numeric(data)) stop("data must be a numeric vector")
  if (anyNA(data)) stop("data must not contain NA values")

  periods <- c(short_period, long_period, signal_period)
  if (any(!is.numeric(periods)) || any(is.na(periods)) ||
```

```
    any(periods <= 0) || any(periods != as.integer(periods))) {
  stop("all periods must be positive integers")
 }
 if (short_period >= long_period) {
  stop("short_period must be less than long_period")
 }

 short_ema <- ema(data, short_period)
 long_ema <- ema(data, long_period)
 macd_line <- short_ema - long_ema
 signal_line <- ema(macd_line, signal_period)
 histogram <- macd_line - signal_line

 result <- list(
  macd_line = macd_line,
  signal_line = signal_line,
  histogram = histogram
 )

 return(result)
}
```

6. Standard Deviation (stdev)

The implementation follows the population-standard-deviation formula shown in the assignment: calculate the mean, squared deviations, divide their sum by n, and take the square root. It does not call sd().

```
# BDA400 Assignment 5 - Standard Deviation (stdev)
# Student: Parvana Babali
# Implements the population standard deviation formula from the assignment.

stdev <- function(data) {
 if (!is.numeric(data)) stop("data must be a numeric vector")
 if (length(data) == 0) stop("data must contain at least one value")
 if (anyNA(data)) stop("data must not contain NA values")

 mean_value <- sum(data) / length(data)
 diff_values <- data - mean_value
 squared_diff <- diff_values * diff_values
 variance <- sum(squared_diff) / length(squared_diff)
 standard_deviation <- sqrt(variance)

 return(standard_deviation)
}
```

7. Linear Regression (linreg)

The function extracts exactly regressionLength observations ending regressionOffset observations before the most recent value. It calculates slope and intercept directly from sums of deviations and returns predicted values. The one-based R start index includes +1 so the subset contains exactly regressionLength points.

```
# BDA400 Assignment 5 - Linear Regression (linreg)
# Student: Parvana Babali
# Base R only - the regression parameters are calculated directly.
```

```
linreg <- function(regressionSource, regressionLength, regressionOffset) {
  if (!is.numeric(regressionSource)) {
    stop("regressionSource must be a numeric vector")
  }
  if (anyNA(regressionSource)) stop("regressionSource must not contain NA values")

  n <- length(regressionSource)

  if (length(regressionLength) != 1 || !is.numeric(regressionLength) ||
      is.na(regressionLength) || regressionLength <= 1 ||
      regressionLength != as.integer(regressionLength)) {
    stop("regressionLength must be an integer greater than 1")
  }
  if (regressionLength > n) {
    stop("regressionLength cannot be greater than the number of elements in regressionSource")
  }
  if (length(regressionOffset) != 1 || !is.numeric(regressionOffset) ||
      is.na(regressionOffset) || regressionOffset < 0 ||
      regressionOffset != as.integer(regressionOffset)) {
    stop("regressionOffset must be a non-negative integer")
  }
  if (regressionOffset >= regressionLength) {
    stop("regressionOffset must be less than regressionLength")
  }
  if (regressionOffset + regressionLength > n) {
    stop("regressionLength plus regressionOffset exceeds the available data")
  }

  # In R's one-based indexing, +1 ensures exactly regressionLength values.
  start_index <- n - regressionOffset - regressionLength + 1
  end_index <- n - regressionOffset
  source_subset <- regressionSource[start_index:end_index]

  index_values <- seq_along(source_subset)
  mean_index <- sum(index_values) / length(index_values)
  mean_source <- sum(source_subset) / length(source_subset)

  numerator <- sum((index_values - mean_index) *
                   (source_subset - mean_source))
  denominator <- sum((index_values - mean_index)^2)

  if (denominator == 0) stop("cannot calculate regression with zero index variance")

  slope <- numerator / denominator
  intercept <- mean_source - slope * mean_index
  predicted_values <- slope * index_values + intercept

  result <- list(
    slope = slope,
    intercept = intercept,
    predicted_values = predicted_values
  )

  return(result)
}
```

8. Relative Strength Index (RSI)

The implementation calculates consecutive changes, separates gains and losses, initializes average gain/loss across the first period, then applies Wilder smoothing. It handles zero-loss and zero-gain cases explicitly and returns an RSI vector aligned to the input data.

```
# BDA400 Assignment 5 - Relative Strength Index (RSI)
# Student: Parvana Babali
# Uses Wilder's smoothing and no external libraries.

rsi <- function(data, period) {
 if (!is.numeric(data)) stop("data must be a numeric vector")
 if (anyNA(data)) stop("data must not contain NA values")
 if (length(period) != 1 || !is.numeric(period) || is.na(period) ||
     period <= 0 || period != as.integer(period)) {
  stop("period must be one positive integer")
 }
 if (length(data) <= period) {
  stop("data length must be greater than period")
 }

 diff_values <- diff(data)
 gains <- ifelse(diff_values > 0, diff_values, 0)
 losses <- ifelse(diff_values < 0, abs(diff_values), 0)

 avg_gain <- sum(gains[1:period]) / period
 avg_loss <- sum(losses[1:period]) / period

 rsi_values <- rep(NA_real_, length(data))

 calc_rsi <- function(gain, loss) {
  if (gain == 0 && loss == 0) return(50)
  if (loss == 0) return(100)
  if (gain == 0) return(0)
  rs <- gain / loss
  100 - (100 / (1 + rs))
 }

 rsi_values[period + 1] <- calc_rsi(avg_gain, avg_loss)

 if (length(data) > period + 1) {
  for (i in (period + 2):length(data)) {
   avg_gain <- (avg_gain * (period - 1) + gains[i - 1]) / period
   avg_loss <- (avg_loss * (period - 1) + losses[i - 1]) / period
   rsi_values[i] <- calc_rsi(avg_gain, avg_loss)
  }
 }

 return(rsi_values)
}
```

9. Stochastic RSI (StochRSI)

The function first calls the student-written rsi(), normalizes the valid RSI values between their minimum and maximum, then calculates the %K line with the student-written sma() and the %D line as another SMA.

```
# BDA400 Assignment 5 - Stochastic RSI (StochRSI)
```

```
# Student: Parvana Babali
# Depends only on the student-written rsi() and sma() functions.

stoch_rsi <- function(data, period, k_period, d_period) {
  periods <- c(period, k_period, d_period)
  if (any(!is.numeric(periods)) || any(is.na(periods)) ||
      any(periods <= 0) || any(periods != as.integer(periods))) {
    stop("period, k_period, and d_period must be positive integers")
  }

  rsi_values <- rsi(data, period)
  valid_rsi <- rsi_values[!is.na(rsi_values)]

  if (length(valid_rsi) < k_period) {
    stop("not enough RSI values for k_period")
  }

  min_rsi <- min(valid_rsi)
  max_rsi <- max(valid_rsi)

  if (max_rsi == min_rsi) {
    normalized_rsi <- rep(0, length(valid_rsi))
  } else {
    normalized_rsi <- (valid_rsi - min_rsi) / (max_rsi - min_rsi)
  }

  k_line <- sma(normalized_rsi, k_period)

  if (length(k_line) < d_period) {
    stop("not enough %K values for d_period")
  }

  d_line <- sma(k_line, d_period)

  result <- list(
    k_line = k_line,
    d_line = d_line
  )

  return(result)
}
```

10. Crossover

The function compares two equal-length arrays and records Up when arr1 moves from at-or-below arr2 to above it. Following the assignment pseudocode, it also records Down when the opposite crossing occurs and None otherwise.

```
# BDA400 Assignment 5 - Crossover
# Student: Parvana Babali
# Returns Up, Down, or None for each position, matching the assignment pseudocode.

crossover <- function(arr1, arr2) {
  if (!is.numeric(arr1) || !is.numeric(arr2)) {
    stop("arr1 and arr2 must be numeric vectors")
  }
  if (length(arr1) != length(arr2)) {
    stop("Both arrays should have the same length")
```

```
 }
 if (length(arr1) == 0) return(character(0))

 crossover_signals <- rep("None", length(arr1))

 if (length(arr1) > 1) {
  for (i in 2:length(arr1)) {
   if (anyNA(c(arr1[i - 1], arr1[i], arr2[i - 1], arr2[i]))) {
    crossover_signals[i] <- "None"
   } else if (arr1[i] > arr2[i] && arr1[i - 1] <= arr2[i - 1]) {
    crossover_signals[i] <- "Up"
   } else if (arr1[i] < arr2[i] && arr1[i - 1] >= arr2[i - 1]) {
    crossover_signals[i] <- "Down"
   } else {
    crossover_signals[i] <- "None"
   }
  }
 }

 return(crossover_signals)
}
```

11. Crossunder

The crossunder function records True when arr1 moves from at-or-above arr2 to below it, False when that event does not occur, and None at the first position because no previous point exists.

```
# BDA400 Assignment 5 - Crossunder
# Student: Parvana Babali
# Returns True/False strings as specified in the assignment pseudocode.

crossunder <- function(arr1, arr2) {
 if (!is.numeric(arr1) || !is.numeric(arr2)) {
  stop("arr1 and arr2 must be numeric vectors")
 }
 if (length(arr1) != length(arr2)) {
  stop("Both arrays should have the same length")
 }
 if (length(arr1) == 0) return(character(0))

 crossunder_signals <- rep("False", length(arr1))
 crossunder_signals[1] <- "None"

 if (length(arr1) > 1) {
  for (i in 2:length(arr1)) {
   if (anyNA(c(arr1[i - 1], arr1[i], arr2[i - 1], arr2[i]))) {
    crossunder_signals[i] <- "False"
   } else if (arr1[i] < arr2[i] && arr1[i - 1] >= arr2[i - 1]) {
    crossunder_signals[i] <- "True"
   } else {
    crossunder_signals[i] <- "False"
   }
  }
 }

 return(crossunder_signals)
}
```

12. Validation and Expected Outputs

The functions were independently cross-checked using the same formulas and sample data. The table below records representative expected results used for verification.

Function	Sample Call	Expected / Key Result
sma	sma(c(10,12,15,20,18,22,25,24,21), 3)	First value = 12.333333; 7 SMA values total
ema	ema(c(10,12,15,20,18,22,25,24,21), 3)	Last EMA = 22.078125
macd	macd(c(100,105,110,115,120,125,130),3,5,2)	Last histogram = 0.216800
stdev	stdev(c(10,12,15,20,18,22,25,24,21))	Population SD = 4.946629
linreg	linreg(sample, 5, 0)	Slope = 0.800000; intercept = 19.600000
rsi	rsi(c(45,50,48,55,52,49,58,60,65,62),5)	Last RSI = 70.223909
stoch_rsi	stoch_rsi(data4,5,3,3)	%K count = 13; %D count = 11
crossover	crossover(arr1, arr2)	None, None, None, Up, None, None, None, None, None
crossunder	crossunder(arr1, arr2)	None, False, False, False, False, False, False, False, False

13. Code Accuracy and Robustness

- All required function names and argument signatures follow the assignment templates.
- No technical-analysis indicator library is used in the nine implementation files.
- Input validation is included for numeric data, valid periods, array lengths, offsets, and missing values.
- MACD reuses the student-written EMA function; StochRSI reuses the student-written RSI and SMA functions.
- The test script includes known-value assertions for SMA, standard deviation, crossover, and crossunder.
- The integration script applies all indicators to an Assignment 2-style Adjusted or Close price column.

14. Repository and Submission

Repository: https://github.com/ParvanaB/TechnicalAnalysis

Recommended location for these files: TechnicalAnalysis/Assignment05/. Upload the separate R scripts and this report to that folder. The LMS PDF is named Babali_Parvana_CA_BDA400_A05.pdf.

Appendix A - Test Script

```
# BDA400 Assignment 5 - Validation Test Script
# Student: Parvana Babali

source("sma.R")
source("ema.R")
source("macd.R")
source("stdev.R")
source("linreg.R")
source("rsi.R")
source("stoch_rsi.R")
source("crossover.R")
source("crossunder.R")

cat("=== SMA ===\n")
data1 <- c(10, 12, 15, 20, 18, 22, 25, 24, 21)
print(sma(data1, 3))

cat("\n=== EMA ===\n")
print(ema(data1, 3))

cat("\n=== MACD ===\n")
data2 <- c(100, 105, 110, 115, 120, 125, 130)
print(macd(data2, 3, 5, 2))

cat("\n=== STANDARD DEVIATION ===\n")
print(stdev(data1))

cat("\n=== LINEAR REGRESSION ===\n")
print(linreg(data1, regressionLength = 5, regressionOffset = 0))

cat("\n=== RSI ===\n")
data3 <- c(45, 50, 48, 55, 52, 49, 58, 60, 65, 62)
print(rsi(data3, 5))

cat("\n=== STOCHASTIC RSI ===\n")
data4 <- c(45, 50, 48, 55, 52, 49, 58, 60, 65, 62,
        67, 70, 68, 72, 75, 73, 77, 80, 78, 82)
print(stoch_rsi(data4, period = 5, k_period = 3, d_period = 3))

cat("\n=== CROSSOVER ===\n")
arr1 <- c(10, 12, 15, 20, 18, 22, 25, 24, 21)
arr2 <- c(18, 20, 22, 18, 15, 12, 10, 11, 13)
print(crossover(arr1, arr2))

cat("\n=== CROSSUNDER ===\n")
print(crossunder(arr1, arr2))

# Simple assertions for known values.
stopifnot(all.equal(sma(data1, 3)[1], 37 / 3))
stopifnot(abs(stdev(c(1, 2, 3)) - sqrt(2 / 3)) < 1e-12)
stopifnot(crossover(c(1, 3), c(2, 2))[2] == "Up")
stopifnot(crossunder(c(3, 1), c(2, 2))[2] == "True")

cat("\nAll basic validation checks completed.\n")
```

Appendix B - Independent Validation Log

BDA400 Assignment 5 - Independent Validation Summary
Student: Parvana Babali

SMA(3) expected: 12.333333, 15.666667, 17.666667, 20.000000, 21.666667, 23.666667, 23.333333
EMA(3) expected: 10.000000, 11.000000, 13.000000, 16.500000, 17.250000, 19.625000, 22.312500, 23.156250, 22.078125
MACD line expected: 0.000000, 0.833333, 1.805556, 2.662037, 3.337191, 3.839378, 4.200210
MACD signal expected: 0.000000, 0.555556, 1.388889, 2.237654, 2.970679, 3.549811, 3.983410
MACD histogram expected: 0.000000, 0.277778, 0.416667, 0.424383, 0.366512, 0.289566, 0.216800
Population stdev expected: 4.946629
Linreg slope expected: 0.800000
Linreg intercept expected: 19.600000
Linreg predictions expected: 20.400000, 21.200000, 22.000000, 22.800000, 23.600000
RSI expected: NA, NA, NA, NA, NA, 60.000000, 74.400000, 76.727273, 81.876106, 70.223909
StochRSI %K expected: 0.469805, 0.799982, 0.736952, 0.741571, 0.719227, 0.740807, 0.751057, 0.766712, 0.764214, 0.770924, 0.780770, 0.779237, 0.783187
StochRSI %D expected: 0.668913, 0.759501, 0.732583, 0.733868, 0.737031, 0.752859, 0.760661, 0.767283, 0.771969, 0.776977, 0.781065
Crossover expected: None, None, None, Up, None, None, None, None, None
Crossunder expected: None, False, False, False, False, False, False, False, False